

1. Архитектура, микроархитектура: отличия, примеры.....	3
2. Производительность процессора: понятие “производительность”, способы измерения, единицы измерения.....	4
3. Классификация архитектур (CISC, RISC, VLIW, EPIC).....	5
4. Классификация архитектур (x86, x86-64, Power, ARM, IA64, RISC-V, MIPS, Alpha, ...)	8
5. Кодирование инструкций на примере MIPS и x86	11
6. Типы инструкций. Примеры	13
7. Арифметические инструкции. Инструкции вещественного со- процессора	14
8. Регистровый файл. Разновидности регистрового файла	16
9. Организация памяти. Устройство управления памятью (MMU)	18
10. Виртуальная память. TLB.....	21
11. Алгоритм выбора строки жертвы, запись данных в кэш, обработка кэш промаха при записи.....	22
12. Кэш: классификация. Пространственная и временная локальность.....	23
13. Кэш: логическая организация, ассоциативность. Поиск данных в КЭШе.....	24
14. Кэш: протоколы когерентности	27
15. Кэш: способы оптимизации ПО, prefetching.....	28
1. Общие способы оптимизации кэша в ПО	28
2. Prefetching: предзагрузка данных в кэш.....	28
16. Векторные архитектуры. Примеры	29
17. Векторизация. Векторные инструкции. Развертка циклов.....	30
18. SIMD-инструкции. Примеры.....	32
19. Конвейер. Характеристики конвейера. Конвейер инструкций	33
20. Предсказание переходов. Локальный предсказатель.....	34
21. Конфликты исполнения инструкций на конвейере. Типы конфликтов	36
22. Конфликт по управлению, способы устранения, слот ожидания.....	37
23. Статическое и динамическое планирование инструкций на примере VLIW архитектуры	39
24. Механизмы ускорения выборки инструкций (внеочередное исполнение инструкций, переименование регистров, технологии микро- и макро-fusion).....	41
25. Спекуляция. Спекулятивный суперскалярный процессор. Основные этапы исполнения инструкций.....	43
26. Алгоритм Томасуло. Планирование инструкций	46
27. Алгоритм Томасуло. Состав процессора. Этапы исполнения инструкций.....	47
28. Спекулятивный суперскалярный процессор	49

29. EPIC. Механизмы поддержки спекуляции.....	51
30. EPIC. Пакет инструкций – способ явного задания параллелизма уровня команд.....	54
31. Технология OpenMP	55

1. Архитектура, микроархитектура: отличия, примеры

Архитектура (Instruction Set Architecture, ISA) — Набор инструкций, регистров, режимов адресации, организации виртуальной памяти, обработки исключений и прерываний. То есть все то, что представляет процессор со стороны программиста.

Микроархитектура: способ, которым данная архитектура набора команд реализована в процессоре.

ISA — это интерфейс между CPU и программами, которые на этом CPU исполняются. ISA определяет количество регистров, которые видны программисту, доступные инструкции, правила выравнивания и так далее. В целом ISA - абстракция для программиста.

Микроархитектура определяет то, как CPU устроен внутри. Из каких функциональных узлов он состоит, какая иерархия кэшей, как декодируются инструкции, сколько стадий в конвейере и так далее.

1 Микроархитектура, не влияет на семантику и результат исполнения программ, но влияет на скорость их исполнения.

2 Микроархитектура влияет на энергопотребление процессора. Например, Intel запустила линейку процессоров Atom, которая была нацелена на мобильные устройства. Эти CPU реализовали ту же архитектуру x86, но их внутреннее устройство позволяло экономить потребление энергии, что делало возможным их использование в переносимых устройствах.

Микроархитектура программисту не видна, программист работает на уровне ISA.

Если два разных процессора реализуют одну и ту же ISA, то они могут исполнять одни и те же программы. Например, программы можно запускать как на Intel, так и на AMD.

2. Производительность процессора: понятие “производительность”, способы измерения, единицы измерения

Производительность – это характеристика, которая отражает способность процессора выполнять вычисления за определенное время. Она определяется скоростью и эффективностью выполнения инструкций, обработки данных и взаимодействия с другими компонентами системы.

Типичные показатели производительности – время отклика и пропускная способность (объем данных, обрабатываемых за единицу времени.).

Способы измерения:

1) Теоретические методы (оценка на основе технических характеристик):

- тактовая частота;
- количество операций на такт (CPI – Cycles Per Instruction);
- количество ядер и их взаимодействие.

2) Бенчмарки (задачи, служащие эталонным тестом производительности):

- Синтетические (оценивают процессор в искусственных задачах):

- Dhrystone – производительность целочисленных вычислений, измеряемая в DMIPS (Dhrystone millions of instructions per second);
- NBench – для измерения производительности целочисленных вычислений, операций памяти и вычислений с плавающей точкой.
- SPECobfr – оценивает производительность вычислений с плавающей запятой, которые часто используются в научных, инженерных и других вычислительно сложных задачах, таких как моделирование, обработка изображений и симуляции.

Единицы измерения

1) Количество операций в секунду:

- FLOPS (Floating Point Operations Per Second) – операции с плавающей запятой.
- MIPS (Million Instructions Per Second) – миллионы инструкций в секунду.

2) Время выполнения задач. Время выполнения реальных задач, таких как рендеринг, компиляция или математические расчеты.

3) Балл производительности. Результаты тестов в бенчмарках.

Например:

- Dhrystone: Процессор с результатом 5000 DMIPS способен выполнять 5 миллиардов целочисленных операций в секунду.
- NBench: В тесте Integer Index процессор набрал 150 баллов, что указывает на его высокую скорость обработки целочисленных операций.
- SPECobfr: Для задач с плавающей запятой процессор может достичь 40 баллов в Base режиме, что делает его подходящим для научных вычислений.

3. Классификация архитектур (CISC, RISC, VLIW, EPIC)

CISC архитектура (Complex Instruction Set Computer)

- Малое количество регистров.
- Большое количество режимов адресации
- Сложные инструкции
- Переменная длина инструкции

RISC архитектура (Reduced instruction set computer)

- архитектура загрузки сохранения (load-store)
- большое количество универсальных регистров (register-register)
- простые инструкции
- ограниченное число режимов адресации
- инструкции фиксированной длины

Причины перехода от CISC к RISC:

- конвейеризация процессора
- увеличение объема памяти
- совершенствование компилятора

VLIW (Very Long Instruction Word)

Архитектура, которая специально разработана для повышения количества одновременно исполняемых инструкций. При одинаковом количестве одновременно исполняемых инструкций VLIW архитектура позволяет создать значительно более простой процессор, чем RISC архитектура.

Ключевая идея:

1. множество НЕЗАВИСИМЫХ инструкций группируются в ОДИН пакет (VLIW инструкцию)
2. на одном такте выбирается ОДИН пакет
3. каждая инструкция в пакете соответствует строго определенному функциональному устройству (ФУ)
4. между всеми ФУ делится общий регистровый файл
5. типовая длина пакета – 64-1024 бит (в зависимости от числа ФУ)
6. проверка зависимостей и планирование исполнения возлагается на компилятор

Плюсы:

- аппаратная сложности уменьшена
- отсутствуют блок проверки зависимостей
- существенно упрощены блоки планирования исполнения и декодирования инструкций
- как следствие, сниженное энергопотребление и повышение тактовых частот

Минусы:

- высокие требования к компилятору
- увеличенный размер программного кода

- более высокие требования к пропускной способности памяти и регистрового файла
- незапланированные события (например, кэш-промах) приводят к останову конвейера
- нерациональное использование пропускной способности памяти и функциональных устройств (например, операция пор)

Статический VLIW процессор

- особенность: за запуск инструкций отвечает компилятор
- компилятор должен правильно расставить пор-ы
- если инструкция пойдет на исполнение до того момента, как предыдущая инструкция закончится произойдет сбой
- компилятор всегда выбирает максимальную латентность для устройств сослучайной латентностью

недостатки:

- неэффективность кэша
- очень большой размер программы из-за большого количества пор инструкций

Динамический VLIW процессор

- особенность: за запуск инструкций отвечает процессор
- процессор ожидает готовности данных для всех инструкций в пакете и только после этого запускает пакет
- позволяет эффективно использовать кэш
- уменьшает размер программы

Общие проблемы VLIW архитектур

- большой размер скомпилированной программы
- отсутствие бинарной совместимости
- изменение количественного и качественного состава ФУ процессора приводит к бинарной несовместимости с предыдущей версией
- код нужно перекомпилировать для каждого процессора



EPIC (explicit parallel instruction set) – призвана сохранить достоинства VLIW архитектуры и устранить ее недостатки

общие черты VLIW и EPIC:

- возможность параллельного исполнения инструкций задается на уровне архитектуры, а не определяется динамически в процессе исполнения

преимущества EPIC:

- решена проблема бинарной совместимости между различными процессорами

Спекуляция по управлению в EPIC основана на:

- наличии двух типов инструкций: спекулятивных и не спекулятивных
- параллельном исполнении спекулятивных инструкций из двух ветвей
- использовании предикатирования для выбора результат корректной ветви
- использовании третьего состояния регистров для сохранения поведения исключений во время спекуляции

4. Классификация архитектур (x86, x86-64, Power, ARM, IA64, RISC-V, MIPS, Alpha, ...)

Здесь важнее всего понимать, что все эти архитектуры отличаются только структурой инструкций, внутренней схемой соединения элементов, какими то другими конструктивными особенностями и системой команд. Умирали они потому, что у компанией заканчивались деньги/не успевали за конкурентами.

- MIPS (умерло)
- SPARC (умирает)
- Power (все сложно)
- 80x86 (все сложно)
- ARM
- RISC-V

MIPS

Область применения - встраиваемые решения
1981-2021 (после переход на RISC-V)

Совершенствование конвейера:

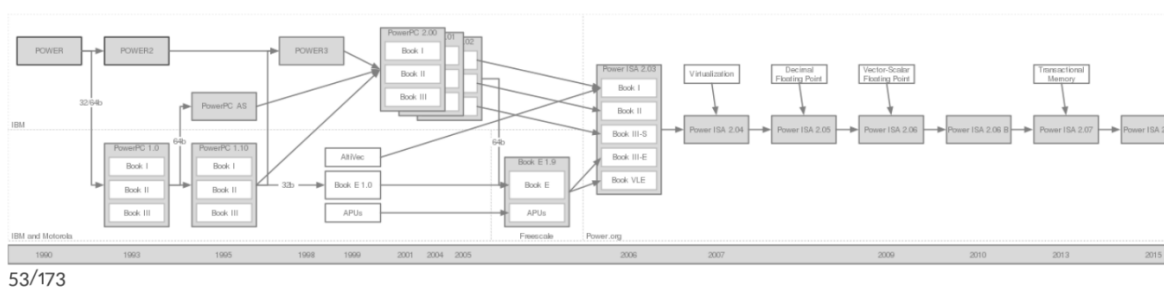
Согласовать каждую подфазу каждой команды (в т.ч. кэширование) в один цикл.

- избегание необходимости в блокировках между стадиями
- пропуск на конвейере только один цикл
- цена: исключение «тяжелых» операций
- исключение регистрового окна

- Малое количество простых инструкций
- Всего 3 формата инструкций (3 способа расположения данных внутри инструкции)
- Широкое распространение в академических кругах
Применялось в сетевых устройствах Cisco, Mikrotik, в старых консолях PlayStation, встраиваемые системы

Power PC

- архитектура POWER
 - Performance Optimization With Enhanced RISC
- область применения
 - высокопроизводительные серверные, настольные, встраиваемые решения
- 1991 г.: альянс AIM: Apple, IBM, Motorola
- 2013 г.: [OpenPOWER Foundation](#)



Юзали древние макбуки, вроде как иногда применяют военной промышленности, инфы очень мало. (здоровая, чё, гробик?)

80x86/x64

Юзают CISC, отличаются в объёме доступной ОЗУ (4 гб для x86, 2⁶⁴ для x86-64), x64 обратно совместима с x86.

В x64 регистры 64 битные, думаю это понятно.

Различные режимы адресации (реальный, защищённый, смешанный).

Расширения SSE1..5, AVX, AVX2, AVX512 и другие подобные.

Производители Intel, AMD, а больше ничего и нет((

- Область применения:
 - высокопроизводительные настольные решения, сервера среднего и начального уровня, суперкомпьютеры
- Основные производители процессоров:
 - Intel, AMD, много мелких компаний из Китая
- В 1978 выпущен 16 битный процессор

ARM (Advanced RISC Machine)

Низкое энергопотребление, поэтому юзаются во встраиваемых решениях. Единое адресное пространство для Ю и ОЗУ

Надо покупать лицензию, чтобы использовать, а так по базе рассказываем про особенности RISC.

- Область применения:
 - встраиваемые решения (смартфоны, планшеты)
- Разработчик:
 - ARM Limited
- Разработка: с 1985 г. (ARM1)
- Семейства:
 - ARM7, ARM9, ARM11, Cortex, Neoverse

Alpha (умер юзал RISC)

- Область применения:
 - высокопроизводительные сервера и суперкомпьютеры
- Основные производители процессоров:
 - DEC -> Compaq (2001) -> HP (до 2007 г.)
- Все процессоры 64-х битные
- С 1992 по 2001 год процессоры Alpha являлись самыми быстрыми в мире
- В 1992 выпущен первый процессор Alpha 21064

PA-RISC (Precision Architecture, умерб юзал RISC)

- область применения:
 - высокопроизводительные сервера и суперкомпьютеры
 - наиболее сложной и развитой системой команд среди всех RISC архитектур
- основные производители процессоров:
 - HP
- в 1989 выпущен первый 32-х битный процессор PA-7000
- в 1996 выпущен 64-х битный процессор PA-8000
- системы сняты с продаж в 2003 году, после перехода HP на процессоры Intel Itanium 2

IA-64 (умер, юзал RISC)

- область применения:
 - высокопроизводительные сервера и суперкомпьютеры
- разработка:
 - Intel + HP (с 1994 г.)
- архитектура относится к новому поколению EPIC
- ядра:
 - Intel Itanium (2001 г.)
 - Intel Itanium 2 (2002-2009г.)
 - Itanium 9700 (2017 г.)

RISC-V:

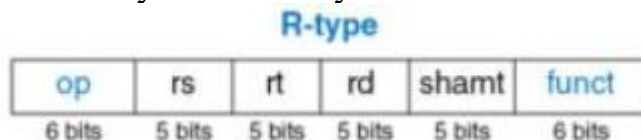
- **Открытая архитектура:** RISC-V — это открытая архитектура (open-source), в отличие от большинства других RISC-архитектур, таких как ARM и MIPS, которые имеют лицензируемые патенты. Это позволяет производителям создавать свои собственные процессоры, используя RISC-V без платы за лицензии.
- **Модульность:** RISC-V отличается модульной структурой, что означает, что его можно настроить под конкретные потребности. Например, можно использовать базовую инструкцию (RV32I или RV64I для 32-битных или 64-битных систем) и добавлять дополнительные расширения для нужд конкретного устройства (например, для работы с плавающей запятой или для криптографии).
- **Развитие и поддержка:** RISC-V активно развивается и поддерживается сообществом, в том числе такими крупными организациями, как Google, Western Digital и другие. Это означает, что архитектура постоянно улучшает свою совместимость с новыми технологиями.

5. Кодирование инструкций на примере MIPS и x86

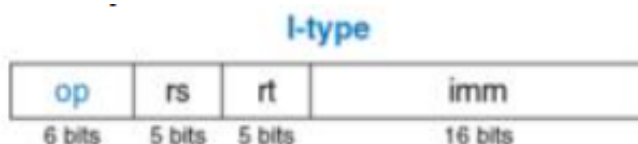
Инструкции бывают:

- постоянной длины
 - простота декодирования
 - RISC: MIPS (4 байта), PowerPC, SPARC, ARM...
- переменной длины
 - занимают меньше памяти
 - CISC: IBM 360, x86(от 1 до 17 байт), Motorola 68k, VAX...
- комбинированный
 - MIPS16, THUMB (2 формата: 2 и 4 байта)

Особенность инструкций MIPS это фиксированная длина(4 байта). В архитектуре MIPS в качестве компромисса используются три формата инструкций: типа R, типа I типа J. Инструкции типа R используют три регистровых операнда. Инструкции типа I используют два регистровых операнда и 16-битную константу. Инструкции типа J (jump) используют 26-битную константой.



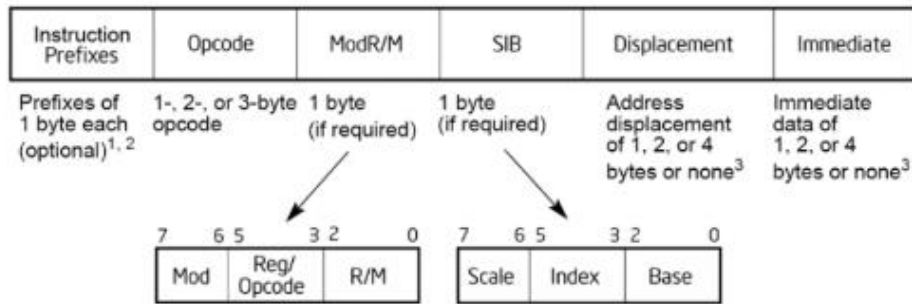
Операция, выполняемая командой, закодирована двумя полями, отмеченными синим цветом: полем op (также называемым opcode или кодом операции) и полем funct (также называемым функцией). У всех команд типа R поле opcode равно нулю. Операнды закодированы тремя полями: rs, rt и rd. Поля содержат номера регистров. Регистры rs и rt являются регистрами-источниками, а rd – регистром-назначением. Поле, shamt, используется только для операций сдвига.



Инструкции типа I используют в качестве операндов два регистра и один непосредственный операнд (константу)



Этот формат используется только для инструкций безусловного перехода и ветвления. 6-бит поле кода операции (opcode). Оставшиеся биты используются для указания адреса перехода (addr).



1. The REX prefix is optional, but if used must be immediately before the opcode; see Section 2.2.1, "REX Prefixes" for additional information.
 2. For VEX encoding information, see Section 2.3, "Intel® Advanced Vector Extensions (Intel® AVX)".
 3. Some rare instructions can take an 8B immediate or 8B displacement.

Главная особенность инструкций x86 – не фиксированная длина инструкций что усложняет архитектуру процессора.

Общий вид инструкции x86:

- Префикс инструкции
- ОPCODE (от 1 до 3 байтов)
- ModR/M - определяет саму операцию
- SIB - байт для операций расширенных (32/64 битных)
- Отклонение - используется для вычисления эффективного адреса инструкции (может быть 1 2 4 байта, его размер определяет поле ModR/M)
- Поле непосредственного операнда имеется в командах, использующих непосредственную адресацию; его наличие определяется по коду операции. Размер этого поля обычно равен размеру операндов данной команды; исключением является 64-разрядный режим

6. Типы инструкций. Примеры

Передача данных

- LD и ST (загрузка и сохранение данных из/в память)
- MFC1 и MTC1 (чтение и загрузка в регистры сопроцессора 1)
- MFC0 и MTC0 (чтение и загрузка в регистры сопроцессора 0)

Арифметико-логические

- ADD и SUB
- AND, OR и XOR
- MUL и DIV
- SLT (Set on Less Than) и LUI (Load Upper Immediate)
- двухадресные (Operand1 = Operand1 OPERATION Operand2) и трехадресные (Operand3 = Operand1 OPERATION Operand2)

Управление потоком исполнения

- BEQZ (Branch if Equal to Zero) – условный переход
- JR (Jump Register) и JAL (Jump and Link) – безусловный переход
- TRAP (генерация програм. исключения) и ERET (Exception Return)
- вызов процедур
- возврат из процедуры
- организация циклов

Для работы с плавающей запятой

- ADD.D (Add Double-Precision) и SUB.S (Subtract Single-Precision)
- MUL.D
- C.LT.D (Compare Less Than, Double-Precision) и CVT.S.W (Convert Word to Single-Precision)

Мультимедийные (SIMD)

- ADD.PS и SUB.PS, MUL.PS, C.LT.PS
- .PS (Paired Single) работают с парами чисел с плавающей точкой одинарной точности в одном регистре

Строковые

- REP (Repeat) MOVSB – копирует последовательность байтов из одной области памяти в другую
- REP MOVSW – копирует последовательность 16-битных слов
- REP MOVSD – копирует последовательность 32-битных слов
- обработка последовательностей данных (строк) в памяти без необходимости написания сложных циклов

Системные

- вызовы ОС
- для работы с виртуальной памятью

Для работы с графикой

- операции для работы с пикселями и вертексами
- операции сжатия/восстановления

7. Арифметические инструкции. Инструкции вещественного со- процессора

1. Типы арифметических операций: сложение, вычитание, умножение, деление.

2. Тип арифметики: циклическая или с насыщением (обработка переполнений путем их циклического сброса (wrap-around) или насыщения (saturation)), знаковая или без знаковая (учитывается или нет знак числа).

- количество операндов

— двухадресные

- Operand1 = Operand1 OPERATION Operand2

— трехадресные

- Operand3 = Operand1 OPERATION Operand2

- разрядность операндов

— для современных 64-ех битных архитектур составляет обычно 32 и 64 бита

— исключение x86/x86-64 (может использовать меньшую разрядность.

Например, 8 или 16)

- адресация операндов

• генерация исключений и модификация флагов состояния процессора (Арифметические инструкции могут изменять флаги процессора (например, Zero, Carry, Overflow))

Инструкции умножения и деления. Проблема операндов.

- Как сохранить удвоенный результат умножения.

1. Результат сохраняется в регистр удвоенной размерности. (например, 128 бит для 64-битных операндов).

2. Старшая часть отбрасывается.

3. Результат сохраняется в два регистра заданных неявно (универсальные или специальные)

4. Для вычисления старшей части операции используется отдельная инструкция.

- Деление.

Для деления процессор может возвращать два результата:

1. Только частное: Часто в быстродействующих системах.

2. Частное и остаток: Сохранение в два регистра, задаваемых неявно.

Инструкции для программной реализации

1) Сдвиг влево и сложение. Инструкция использовалась для умножения на константу.

Пример (умножение на 0xB):

r1 = A

0xB = 1011

shladd r2, r1, r1, 3

shladd r2, r2, r1, 1

2) Инструкции для программной реализации деления.

- DS – divide step. Инструкция вычисляет один бит частного и остаток. Бит частного помещается в carry flag, а остаток в регистр результата.

Инструкции вещественного сопроцессора.

– Инструкции загрузки и сохранения.

- Преобразование входных данных ко внутреннему типу.

– Арифметические инструкции.

- Mull and Add

- Обратное значение.

- Сложные мат. функции (синус, косинус...).

– Инструкции преобразования типов.

– Инструкции сравнения.

Инструкции сравнения

– Результат сравнения помещается во флаги сопроцессора.

Проблемы выполнения переходов на основе флагов сопроцессора

- Копирование флагов сопроцессора в универсальные регистры с их последующим анализом.

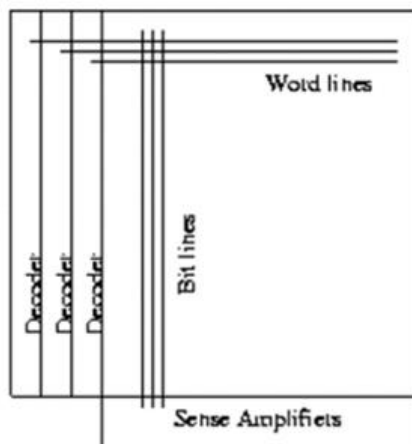
- Инструкции условного перехода, которые вычисляют условие на основе флагов сопроцессора.

- Использование инструкций нуллификаций, которые вычисляют условие на основе флагов сопроцессора.

8. Регистровый файл. Разновидности регистрового файла

Регистровый файл – модуль микропроцессора (CPU), содержащий в себе реализацию регистров процессора. Основные функции: хранение данных, быстрый доступ к ним и управление процессами чтения и записи. Использование регистровых файлов позволяет выполнять внеочередное исполнение машинных инструкций (не по порядку следования в коде, а по их готовности) и переименование регистров для реализации параллелизма на уровне команд.

Реализован как массив запоминающих ячеек, считываемых вертикально.



Слова (данные) расположены в горизонтальных строках (word lines), и при чтении ячейки выдают своё значение на вертикальные битовые линии (Bit lines). В нижней части эти линии подключены к усилителям, которые преобразуют сигналы с ячеек в булевы сигналы полной амплитуды. В левой части расположены декодеры, активирующие строку, соответствующую заказанному регистру.

Классический регистровый файл

- Количество регистров: 8 – 32
- Разрядность регистров: 32 или 64 бита

+: быстрый

-: программное сохр. требуемых регистров при вызове процедур -> увеличение задержки при переключении

Стековый регистровый файл

- Используется в архитектуре x86.
- Позволяет неявно обращаться к операндам на вершине стека.
- Используется в архитектуре вещественных сопроцессов SPARC и PA-RISC

Оконный регистровый файл

- Используется в архитектуре SPARC.
- Большое количество регистров: до 640
- Доступ к регистрам осуществляется через окно фиксированного размера: 32 регистра.
- Окно разбито на 4 части: Глобальные регистры (неизменны при переключении окон), Локальные регистры (для локальных данных), Собственные входные/выходные параметры, Входные/выходные параметры вызываемой функции.
- Регистр CWP (Current Window Pointer) – указывает на активное в данный момент окно
- Альтернативные глобальные регистры – гибкость при переключении окон

Для организации одного локального окна требуется 16 регистров. При переполнении регистрового файла генерируется исключение, по которому ОС может очистить несколько регистровых окон.

+: Передача параметров между процедурами происходит через регистры. Можно избежать необходимости сохранять регистры в память

-: Низкая скорость регистрового файла; генерация исключения при переполнении регистрового файла; недостаточная гибкость (фиксированный размер окна, фиксированное количество локальных регистров, и регистров, которые используются для передачи параметров)

Динамический оконный регистровый файл (регистровый стек)

- Используется в архитектуре IA-64
- Регистровое окно произвольного размера
- Количество локальных регистров и регистров входных/выходных параметров произвольно
- Использование механизма Register Stack Engine (RSE) для автоматического сохранения/загрузки регистров в случае переполнения регистрового файла
- Количество глобальных регистров: 32, стековых: 96

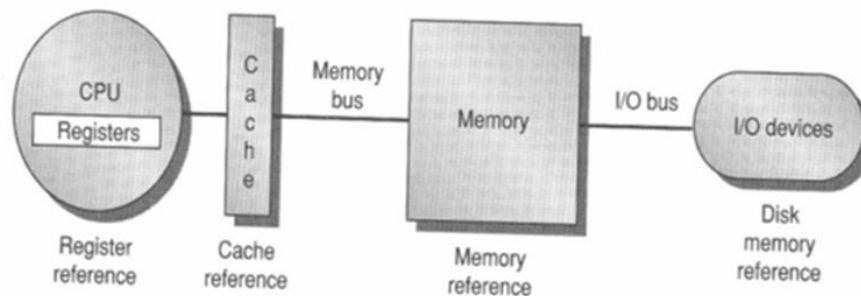
+: ПО не нужно заботиться о сохранении/загрузке регистров при вызове процедур; эффективное использование регистров из-за изменяемого размера окна

-: Высокая сложность аппаратной реализации RSE

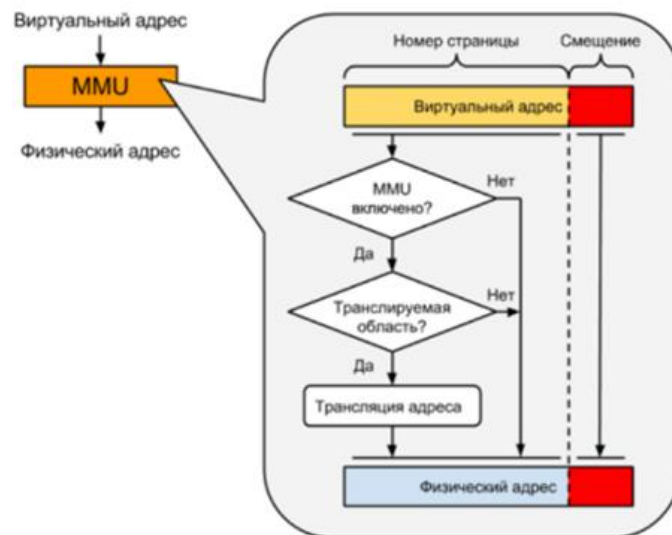
9. Организация памяти. Устройство управления памятью (MMU)

Память организована в виде иерархии. Типичная иерархия памяти выглядит следующим образом (снизу вверх):

1. Регистр процессора – самый быстрый, но ограниченный объём памяти, используемый для хранения данных и инструкций, с которыми работает процессор.
2. Кэш-память – быстрый тип памяти, расположенный между процессором и основной памятью. Кэш бывает разных уровней:
 - **L1** – маленький и быстрый, обычно встроен непосредственно в процессор.
 - **L2** – немного больше и медленнее, часто находится рядом с процессором.
 - **L3** – ещё более ёмкий, но медленный по сравнению с L1 и L2, может быть разделён между несколькими процессорами.
3. Оперативная память (RAM) – основная память системы, куда загружаются данные и программы, с которыми работает компьютер в данный момент. Это более медленный, но большой тип памяти.
4. Внешняя память – хранилище данных на длительный срок. Медленнее, чем RAM, но имеет гораздо большую ёмкость.



MMU (Memory Management Unit) - специализированное аппаратное устройство, которое управляет доступом к памяти и отвечает за преобразование виртуальных адресов в физические. +: использование при необходимости; отсутствие влияния на CPU.



Основные задачи:

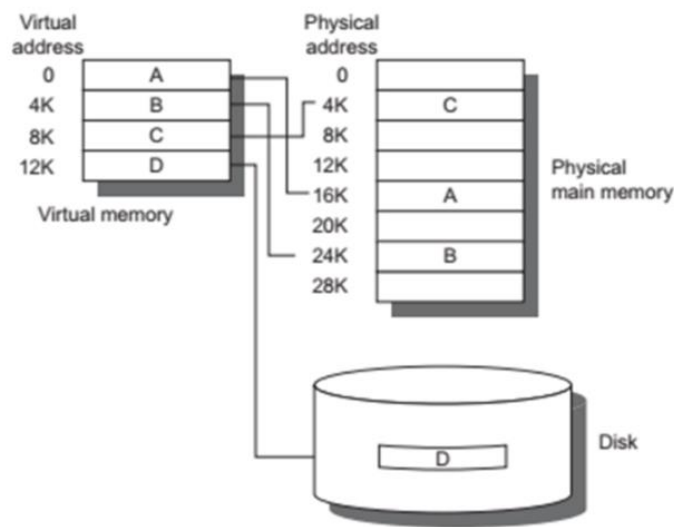
1. **Преобразование виртуальных адресов в физические** для работы с виртуальной памятью, которая позволяет процессам работать с абстракцией памяти, как если бы у каждого процесса был свое изолированное адресное пространство. Но на самом деле процессы используют одну физическую память.
2. **Разделение памяти между процессами: MMU** обеспечивает защиту и изоляцию процессов, чтобы они не имели доступа к ресурсам друг друга.
3. **Управление страницами памяти (paging) и сегментами:** для эффективного использования памяти. MMU управляет переводом виртуальных страниц в физические страницы. Это позволяет выделять память динамически и загружать в память только те части программы, которые необходимы в данный момент.
4. **Защита памяти:** проверка, имеет ли процесс право на доступ к конкретному диапазону памяти.
5. **Использование кэш-памяти для ускорения доступа**

10. Виртуальная память. TLB

Виртуальная память - концепция, которая позволяет уйти от использования физических адресов, используя вместо них виртуальные.

+: расширение реального адресного пространства; создание изолированных адресных пространств для различных процессов; задание различных свойств для разных участков памяти

- все физическое адресное пространство процесса разбито на виртуальные страницы
- вся физическая память разбита на физические страницы такого же размера
- у каждого процессора существует таблица, которая связывает виртуальные страницы с физическими страницами
- кроме оперативной памяти и страницы могут находиться в файле подкачки



Оперативная память выступает в качестве кэша для «бесконечной» дисковой памяти; для вытеснения страниц из памяти используется LRU-алгоритм (удаление элементов, которые не использовались долгое время); за перенос страниц между диском и ОЗУ отвечает ОС. При отсутствии требуемой страницы в ОЗУ процессор генерирует исключение. При вытеснении страниц из памяти только модифицированные страницы сохраняются на диск.

TLB (translation look-aside buffer) - это кэш, используемый в устройстве управления памятью (MMU) для ускорения процесса преобразования виртуальных адресов в физические. Он хранит недавние преобразования виртуальных адресов (или страниц) в физические.

Когда процессор обращается к памяти, он генерирует виртуальный адрес. MMU, получив этот виртуальный адрес, должна преобразовать его в физический. Это преобразование осуществляется с использованием таблицы страниц. Для ускорения используется TLB.

Свойства: На вход TLB приходит виртуальный адрес страницы. Элемент TLB - дескриптор страницы. Процессор содержит два TLB: один для данных, второй для инструкций. Кэш TLB обычно полностью ассоциативный. Может иметь несколько уровней. При переключении процесса TLB очищается.

11. Алгоритм выбора строки жертвы, запись данных в кэш, обработка кэш промаха при записи

Когда кэш заполняется, необходимо выбрать строку (line) для замены. Стратегии выбора строки жертвы:

- Случайная. Удаляется случайная строка из множества.
- FIFO. Удаляется строка, которая дольше всех находится в кэше.
- LRU (Least-recently used). Удаляется строка, к которой реже всего обращаются.
- Замена наименее часто использовавшейся строки (LFU — Least Frequently Used). Заменяется та строка в кэш-памяти, к которой было меньше всего обращений.

Запись в кэш

1 Операции записи встречаются в 5 раз реже операций чтения.
2 Операция записи начинается, только после того, как окончательно выяснено есть ли нужная строка в кэше или нет.

Две политики записи в кэш:

– Сквозная запись (write through). Одновременно выполняется запись данных как в кэш, так и в основную память.

– Обратная запись (write back). Первоначально запись производится только в кэш. Все кэш линии, в которые происходила запись помечаются специальным флагом. Если такая строка выталкивается из кэша, то производится ее запись в память целиком.

Обработка кэш промаха при записи

1 При использовании сквозной записи в память, строка в кэш памяти не выделяется, а сразу производится запись данных в память.

2 При использовании обратной записи в память:

– В кэше выделяется пустая строка, с расчетом на то, что она будет полностью заполнена данными.

– Строка загружается из памяти, а затем происходит ее модификация.

12. Кэш: классификация. Пространственная и временная локальность

Разновидности кэша

- инклюзивный (следующий уровень иерархии включает все данные предыдущего уровня)
- эксклюзивный (нет общих данных между уровнями)

По политике управления

- write-through (изменения в кэше сразу записываются в основную память)
- write-back (данные записываются в основную память только при вытеснении из кэша)

По технологии организации

- с прямым отображением direct mapping (каждый блок в основной памяти отображается на конкретную ячейку в кэше)
- полностью ассоциативная fully associative mapping (каждое место в кэше может содержать данные из любой области памяти)
- set associative mapping (гибрид direct mapping и fully associative mapping; кэш делится на несколько множеств (sets), и каждый блок памяти может быть размещен в любом из этих множеств, но только в одном месте)

Пространственная и временная локальность программ и данных

- пространственная локальность состоит в том, что если текущее обращение к памяти произошло по адресу «А», то следующее обращение вероятно произойдет по соседнему адресу.
- временная локальность состоит в том, что если произошло обращение к памяти по адресу «А», то вероятно, что в ближайшее время произойдет еще одно обращение по этому адресу

13. Кэш: логическая организация, ассоциативность.

Поиск данных в КЭШе

Кэш – высокоскоростная статическая память, расположенная на процессорном кристалле или в процессорном корпусе.

Иерархия памяти.

Кэш организуется в несколько уровней:

L1: Самый быстрый и маленький уровень кэша, обычно разделенный на данные и инструкции.

L2: Более объемный, но медленный по сравнению с L1.

L3: Общий для всех ядер процессора, самый объемный, но медленный в иерархии кэшей.

Организация.

Весь объем кэша разбит на блоки или кэш линии. Кроме данных в кэше, для каждой кэш линии хранится ее тэг. Тэг кэш линии хранит старшие биты адреса блока памяти, который в ней содержится.

Ассоциативность.

Соответствие между блоком кэша и блоками памяти определяет понятие *ассоциативности*.

- Кэш с прямым отображением (direct mapping) устроен так, что каждая строка ячеек основной памяти может отображаться только в одной, фиксированной для неё строке кэша (под строкой понимается минимальная порция информации, которая может передаваться в кэш). Этот тип обладает минимальным временем поиска, но и минимальной вероятностью присутствия необходимой информации в нём.

- Полностью ассоциативный кэш (fully associative mapping), наоборот, позволяет каждой строке основной памяти отображаться в произвольном месте кэша. Этот кэш, наоборот, обеспечивает максимальную вероятность успеха запросов CPU на требуемую информацию, но при максимальном времени поиска.

- Множественно-ассоциативный кэш (set associative mapping) является сочетанием первых двух типов. Кэш этого типа разделён на фиксированное количество областей, именуемое степенью ассоциативности, и каждая строка RAM может отображаться в произвольном месте только одной из областей кэша. И он позволяет добиться оптимальной производительности. Важным параметром для множественно-ассоциативного кэша является правильный выбор степени ассоциативности или, что то же самое, количества каналов у кэша второго уровня.

Поиск данных в КЭШе

1) Кэш с прямым отображением.

Этапы поиска в кэше:

1. Извлекается средняя часть адреса ($\log_2 m$), определяющая номер кэш-линии в кэше.

2. Тэг кэш-линии с данным номером сравнивается со старшей частью адреса ($\log_2 N$).

Результат:

Если было совпадение по одному из тэгов, то произошло кэш-попадание.

Если не было совпадения ни по одному из тэгов, то произошел кэш-промах.

2) Полностью ассоциативный кэш.

Этапы поиска в кэше:

1. Тэги всех кэш-линий сравниваются со старшей частью адреса одновременно.

Результат, как и в первом случае.

3) Множественно-ассоциативный кэш.

Этапы поиска в кэше:

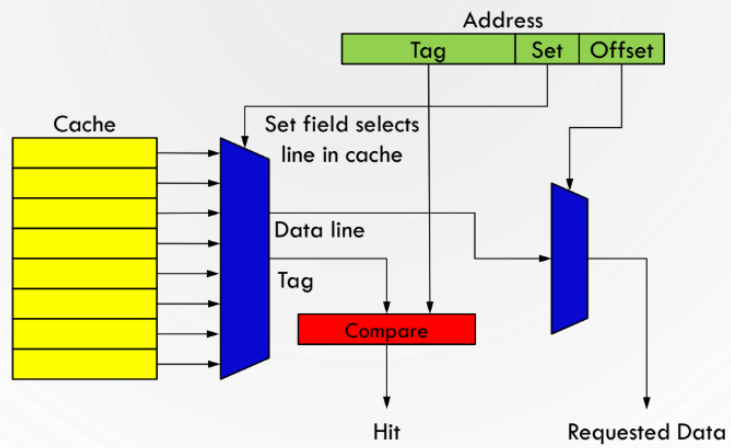
1. Извлекается средняя часть адреса ($\log_2 m$), определяющая номер сета в кэше.

2. Тэги всех кэш-линий данного сета сравниваются со старшей частью адреса ($\log_2 N$) одновременно.

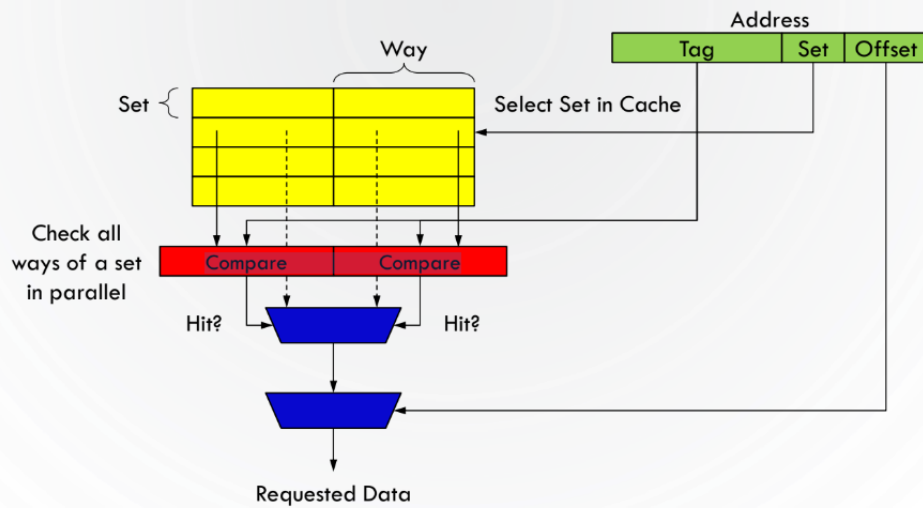
Результат, как и в первом случае.



ПРЯМОЕ ОТОБРАЖЕНИЕ



МНОЖЕСТВЕННО-АССОЦИАТИВНОЕ ОТОБРАЖЕНИЕ (КП)



14. Кэш: протоколы когерентности

Когерентность кэша — свойство кэшей, означающее целостность данных, хранящихся в локальных кэшах для разделяемого ресурса.

Когерентность кэшей — частный случай когерентности памяти.

Когда процессы в системе используют кэширование общих ресурсов, например, памяти, могут возникнуть проблемы с противоречивостью данных. Это особенно справедливо в отношении процессоров в многопроцессорной системе. Когерентность кэшей предназначена для управления такими конфликтами путём поддержания согласованности данных в разных кэшах.

Некогерентные кэши: значение одной ячейки памяти разл3 состояния:

Modified: блок изменен в кэше и не соответствует в ОЗУ. Только один кэш в текущий момент может быть в данном состоянии

Shared: блок не изменен и существует как минимум в одном из кэшей в режиме только чтения

Invalid: блок не существует

Протокол MESI

4 состояния:

Exclusive: значение есть только в этом кэше, и оно пока не было изменено (Это состояние преимущество над MSI т.к не надо проверять актуальность данных в других ядрах процессорно для двух кэшей).

Когерентность определяет поведение чтений и записей в одно и то же место памяти. Кэш называется когерентным, если выполняются следующие условия:

- информирование о записи: изменение данных в любом из кэшей должно быть распространено на другие копии этих данных (этой линии кэша) в соседних кэшах;
- сериализация транзакций: операции чтения и записи в одну и ту же ячейку памяти должны быть видимы для всех процессоров в одинаковом порядке.

Протокол MSI

рах)

Протокол MESIF

идея: попытаться получить данные от других уровней кэша

проблема: состояние S - сообщения приходят от всех

решение: F(orwarded) – разновидность состояния S – «первый среди равных».

Состояние получает всегда последний запросивший данные

Протокол MOSI

Проблема MSI (данные в модифицированном состоянии):

отмена транзакции -> сброс данных в основную память -> смена состояния на Invalid -> перезапрос копии

Решение: O(wned) – состояние указывает, что данный кэш владеет эксклюзивными данными (отличие от MSI: в памяти данные не актуальны !!!)

MSI / MOSI: состояние O - оптимизация сброса данных в память (write-back)

MSI / MESI: состояние E - оптимизация числа транзакций, необходимых для чтения/записи от другого процессора (ядра)

15. Кэш: способы оптимизации ПО, prefetching

1. Общие способы оптимизации кэша в ПО

1 Пространственная локальность (Spatial Locality): Использование данных, которые находятся рядом друг с другом в памяти. Например, считывание массивов последовательными блоками, а не случайными элементами.

2 Временная локальность (Temporal Locality): Повторное использование данных, которые недавно были доступны. Например, использование переменных или структур данных из кэша вместо повторного чтения из основной памяти.

3 Избегать случайного доступа к памяти: Не использовать структуры данных с низкой предсказуемостью доступа, например, хэш-таблицы, если это возможно.

4 Минимизирование кэш-промахов: Использование данных, которые уже находятся в кэше.

2. Prefetching: предзагрузка данных в кэш

Prefetching — это механизм, который загружает данные в кэш заранее, до их фактического использования. Это снижает задержки доступа к памяти.

1 Аппаратный prefetching: Выполняется на уровне процессора. Процессор анализирует паттерны доступа к памяти и автоматически предзагружает данные в кэш. Эффективен для предсказуемых последовательностей доступа.

2 Программный prefetching: Вызывается из кода с использованием специальных инструкций или методов. Позволяет разработчику явно указать, какие данные нужно загрузить.

16. Векторные архитектуры. Примеры

Векторизация – это генерация вместо скалярных инструкций векторных инструкций. Векторными называются те инструкции, операндами которых являются вектора. Процесс векторизации может выполняться компилятором или человеком. Возможности векторизации компилятором ограничены, так как сложно на языке высокого уровня представить большинство векторных операций

- Архитектура x86 (x86-64): MMX, SSE-SSE4, SSSE3, 3DNow!
- Архитектура PowerPC: AltiVec, SPE
- Архитектура SPARC: VIS
- Эти расширения объединяет, то что размер вектора для каждой из них равен 128 бит
- 2010 Выпуск AVX расширения для x86, размер вектора 256 бит.

```
for (; i + 15 < size; i += 16) {  
    // Загружаем 16 элементов из массивов a и b в регистры AVX-512  
    __m512 vec_a = _mm512_loadu_ps(&a[i]);  
    __m512 vec_b = _mm512_loadu_ps(&b[i]);  
    __m512 vec_result = _mm512_add_ps(vec_a, vec_b);  
    _mm512_storeu_ps(&result[i], vec_result);  
}
```

17. Векторизация. Векторные инструкции. Развертка циклов

Векторизация – это генерация вместо скалярных инструкций векторных инструкций. Частный случай автоматического распараллеливания кода.

Может выполняться компилятором или человеком (компилятором понятно что хуже).

Сложности если в коде появляются операции if, многомерные или разреженные матрицы.

Сгау-1 первый векторный процессор.

Векторными называются те инструкции, операндами которых являются вектора. (есть в наборе avx2)

Типы векторных инструкций:

1 Векторизованные - берем условные переменные X0 с 1 источника, Y0 со второго, объединяем их в destination.

2 Горизонтальные - берем условные переменные X1, X0 с 1 источника, Y1, Y0 со второго, в destination записываем как $X1 + X0$ и $Y1 + Y0$.

3 Комбинированные - перемножаем n-е элементы с источников, каждые n и n+1 складываем и записываем в destination. Начинаем с n=0.

Развертка циклов - уменьшается количество исполняемых инструкций связанных с вычислением и проверкой условия.

Вместо:

```
For(int I = 0, I < N, I++)
```

```
    Func()
```

Пишем:

```
For(int I = 0, I < N/2, I++) {
```

```
    Func()
```

```
Func()
```

```
}
```

Необходимые условия:

1 цикл for

2 кол-во итераций не должно зависеть от данных

3 не известно во время компиляции количества итераций цикла

Недостатки:

1 чем больше инструкций, тем выше вероятность кэш-промаха.

2 может не хватить регистров.

Векторизация выполняется на основе цикла, только для операций, которые внутри цикла

Скалярные элементы объединяются в вектора и скалярные операции заменяются векторными.

Условия автоматической векторизации:

1 Операция в цикле

2 Итерации цикла не зависимы

- 3 Число итераций не зависит от данных
- 4 Внутри цикла не должно быть вызова процедур
- 5 Шаг между элементами в памяти - 1.

18. SIMD-инструкции. Примеры

SIMD (англ. Single Instruction stream / Multiple Data stream) – одиночный поток команд и множественный поток данных. Эти системы обычно имеют большое количество процессоров, от 1024 до 16384, которые могут выполнять одну и ту же инструкцию относительно разных данных в жесткой конфигурации. Инструкции SIMD можно найти, в той или иной степени, на большинстве процессоров, в том числе AltiVec фирмы IBM и SPE для PowerPC, HP PA-RISC Multimedia Acceleration eXtensions (MAX), MMX и iWMMXt, SSE, SSE2, SSE3 SSSE3 и SSE4.x компании Intel и так далее.

- Целочисленные векторные инструкции входят в наборы MMX, SSE2, SSE3, SSSE3, SSE4.1

- В качестве аргументов инструкции могут использовать как 64-ех битные регистры MMX, так и 128-ми битные регистры XMM.

- Поддерживается два типа арифметики: с насыщением, без насыщения (циклическая).

- Целочисленный векторный набор характеризуется отсутствием ортогональности к типам данных.

Примеры :

1) **x86/x86-64:**

- **MMX, SSE, SSE2, AVX, AVX-512** — работа с 64-, 128-, 256- и 512-битными регистрами.

2) **ARM:**

- **NEON** — 128-битные SIMD-инструкции.
- **SVE** — масштабируемые векторы до 2048 бит.

3) **PowerPC:**

- **AltiVec** — 128-битные мультимедиа инструкции.

4) **RISC-V:**

- **RVV** — векторы переменной длины.

5) **MIPS:**

- **MSA** — SIMD-инструкции с регистрами 128 бит.

19. Конвейер. Характеристики конвейера. Конвейер инструкций

(в лекциях мало инфы, взял из прошлых лет)

Конвейер - способ организации вычислений, направленный на оптимизацию работы процессора за счет реализации параллелизма: в один момент времени может выполняться несколько инструкций на разных этапах.

1) Конвейер

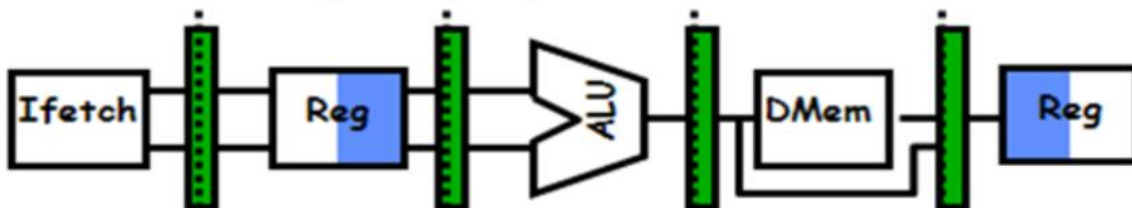
Первый в 1960 UNIVAC США

Характеристики:

- Латентность (время выполнения одной команды на конвейере)
- Пропускная способность (время между поступлением двух команд на конвейер)
- Не конвейер
 - Для быстрого выполнения малого количества операций
- Конвейер
 - Для быстрого выполнения большого количества повторяющихся операций.
 - Время выполнения одной операции больше чем у не конвейерной реализации.

Конвейер инструкций

- Стадии:
 - Выборка инструкций.
 - Выборка данных из регистров и декодирование инструкции.
 - Исполнение инструкции (вычисление адреса).
 - Обращение к памяти (загрузка и сохранение).
 - Запись данных в регистры



Что препятствует эффективному исполнению инструкций на конвейере?

- Зависимости между инструкциями.
- Наличие зависимостей приводит к возникновению конфликтов (hazard).

20. Предсказание переходов. Локальный предсказатель

Предсказание переходов является одним из наиболее эффективных способов борьбы с конфликтами по управлению. Идея заключается в том, что еще до момента выполнения команды условного перехода или же сразу после ее поступления делается предположение о наиболее вероятном результате выполнения такой команды (переход произойдет или нет). Последующие команды подаются на конвейер в соответствии с предсказанием.

При ошибочном предсказании конвейер необходимо вернуть к состоянию, с которого началась выборка ненужных команд, т.е. очистить конвейер, что эквивалентно приостановке конвейера. Цена ошибки может быть достаточно высокой, но при правильном предсказании имеем большой выигрыш – конвейер функционирует ритмично, без остановок. Выигрыш тем больше, чем выше точность предсказания.

Точность предсказания – процентное отношение числа правильных предсказаний к их общему количеству.

Доказано, что для того, чтобы снижение производительности конвейера из-за приостановок по управлению не превысило 10%, точность предсказания должна быть выше 97,7%.

В самом простом случае рассматривается результат самого последнего выполнения команды (однобитовая схема предсказания ветвлений). Биты предсказания ветвлений в однобитовой схеме называют переключателем – “принято/не принято” (taken /not taken).

В состоянии 0 прогнозируется, что переход не выполняется, в состоянии 1 – выполняемый. Сигналы ПВ (переход выполняется) и ПНВ (переход не выполняется) поступают на вход автомата из блока выполнения команды ветвления (Branch unit). С их помощью осуществляется коррекция прогноза.

Локальный предсказатель – результат тот, который для выдачи результата использует только историю своего перехода

Предсказание ветвления:

- Две задачи:
 - Предсказание выполнимости перехода.
 - Предсказание адреса перехода.
- Используется на двух этапах:
 - Выборки инструкций.
 - Планирование и исполнение инструкций.

Обычно под предсказанием инструкции ветвления понимается предсказание выполнимости перехода, так как относительно заданный адрес перехода легко вычислим.

- Предсказатель условного перехода. Два режима работы:
 - Предсказание. На вход адрес инструкции, на выходе результат предсказания.
 - Корректировка. На вход адрес инструкции и направление перехода.

Результат работы – коррекция внутренних состояний.

Однобитная таблица предсказания ветвления:

- Элемент таблицы: однобитная ячейка.
 - Единица – выполнять переход
 - Ноль – пропустить переход
 - Если предсказание неверно, то инвертировать бит.
- Недостатки:
 - Плохо предсказывает поведение коротких циклов. Из-за того, что при выходе из цикла более вероятным становится пропуск перехода.

21. Конфликты исполнения инструкций на конвейере.

Типы конфликтов

Конфликты – это некоторая ситуация, при которой невозможно запустить следующую инструкцию непосредственно за предыдущей, не нарушив критерия сохранения корректности программы.

Запуск инструкции происходит спустя n тактов.

Не атрибут программы или алгоритма, это следствие параллельного исполнения зависимых инструкций на процессоре.

Критерии сохранения корректности программы:

1. Сохранение потока данных:

- поток значений передаваемых между инструкциями производителями и потребителями должен быть сохранен;
- инструкции ветвления делают поток данных динамическим.

2. Сохранение поведения исключений

- порядок, количество и тип генерируемых исключений должен сохраниться.

Типы конфликтов:

1. Структурные конфликты
2. Конфликты по данным - следствие наличия зависимостей по данным именам.
3. Конфликты по управлению - следствие наличия зависимости по управлению.

Способы разрешения конфликтов:

- Программные:
 - обнаружение конфликтов в процессе компиляции и добавление инструкций пор;
 - статическое планирование конвейера.
- Аппаратные:
 - добавление устройства обнаружения конфликтов и останова конвейера;
 - добавление специфичных аппаратных решений.

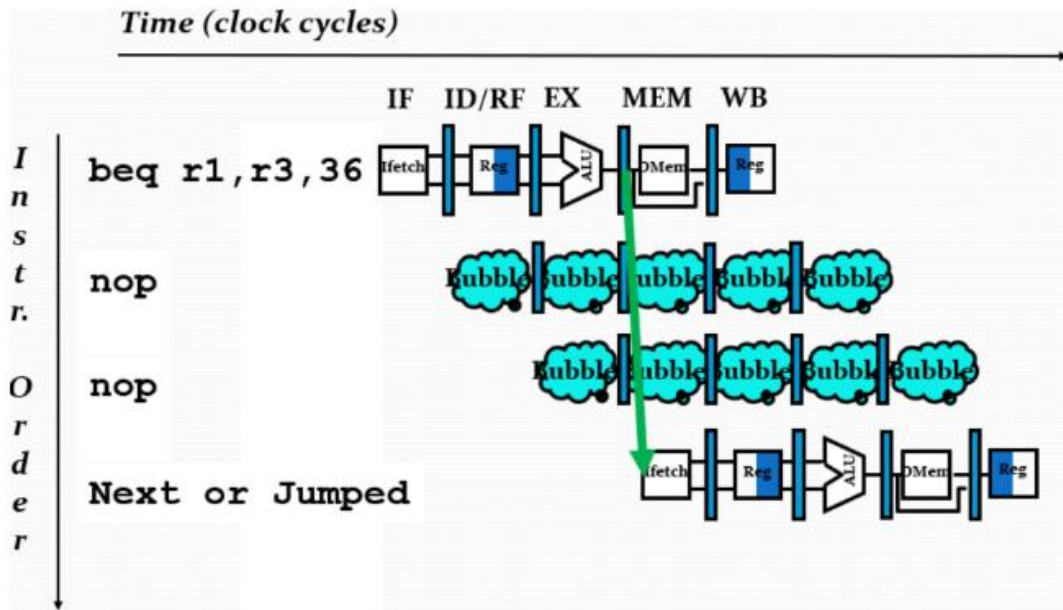
22. Конфликт по управлению, способы устранения, слот ожидания

Конфликты по управлению возникают при конвейеризации команд переходов и других команд, изменяющих значение счетчика команд. Суть конфликтов этой группы наиболее удобно проиллюстрировать на примере команд условного перехода. Пусть в программе команда $i+1$ является командой условного перехода, формирующей адрес следующей команды в зависимости от результата выполнения команды i . Команда i завершит свое выполнение в такте 5. В то же время команда условного перехода уже в такте 3 должна прочитать необходимые ей признаки, чтобы правильно сформировать адрес следующей команды. Если конвейер имеет большую глубину (например, 20 ступеней), то промежуток времени между формированием признака результата и тактом, где он анализируется, может быть еще большим. В инженерных задачах примерно каждая шестая команда является командой условного перехода, поэтому приостановки конвейера при выполнении команд переходов до определения истинного направления перехода существенно скажутся на производительности процессора.

Наиболее эффективным методом снижения потерь от конфликтов по управлению служит предсказание переходов или вставка пор инструкций (менее эффективна). Если после формирования анализируемых признаков оказалось, что направление перехода выбрано верно, все полученные результаты переписываются из буфера по месту назначения, а выполнение программы продолжается в обычном порядке. Если направление перехода предсказано неверно, то буфер результатов очищается. Также очищается и конвейер, содержащий команды, находящиеся на разных этапах обработки, следующие за командой условного перехода. При этом аннулируются результаты всех уже выполненных этапов этих команд. Конвейер начинает загружаться с первой команды другой ветви программы. Так как конвейерная обработка эффективна при большом числе последовательно выполненных команд, то перезагрузка конвейера приводит к значительным потерям производительности.

Конфликт управления

1 Конвейер



Если что, слот ожидания - и есть это время, которое мы заполняем с помощью инструкцию nop(no operation - это ассемблерная инструкция)

23. Статическое и динамическое планирование инструкций на примере VLIW архитектуры

VLIW (Very Long Instruction Word) Исторический обзор архитектур. Способы классификации

- архитектура, специально разработанная для повышения количества одновременно исполняемых инструкций
- позволяет создать более простой процессор, чем RISC архитектура

Ключевая идея VLIW Исторический обзор архитектур. Способы классификации

- дано: конкретный CPU с фиксированным числом функциональных устройств (ФУ)
- принцип работы: — инструкция-пакет простых подинструкций (их кол-во = число ФУ)
— пакет обрабатывается ЦЕЛИКОМ по одному за такт

Статический VLIW процессор

- особенность: за запуск инструкций отвечает компилятор. Компилятор должен правильно расставить пор-ы

* если инструкция пойдет на исполнение до того момента, как предыдущая инструкция закончится произойдет сбой

* как быть с устройствами со случайной латентностью (например, устройство загрузки/сохранения при наличие кэша)

* компилятор всегда выбирает максимальную латентность для устройств со случайной латентностью

недостатки:

- неэффективность кэша
- очень большой размер программы из-за большого количества пор инструкций

Динамический VLIW процессор

Особенность: за запуск инструкций отвечает процессор

- процессор ожидает готовности данных для всех инструкций в пакете.
- только после этого запускает пакет

Позволяет эффективно использовать кэш

Уменьшает размер программы

Пример

Ld\St unit 0	Ld\St unit 1	Integer Alu	Branch Unit
Load R1, A	Load R2, B	Nop	nop
nop	nop	Add R3, R2, R1	nop
Store C,R3	nop	nop	nop

Общие проблемы VLIW архитектур

- Большой размер скомпилированной программы.
- Отсутствие бинарной совместимости.

- изменение количественного и качественного состава ФУ процессора приводит к бинарной несовместимости с предыдущей версией.

- это приводит к тому, что код нужно перекомпилировать для каждого процессора.

24. Механизмы ускорения выборки инструкций (внеочередное исполнение инструкций, переименование регистров, технологии микро- и макро-fusion)

внеочередное исполнение (англ. out-of-order execution)

— исполнение машинных инструкций не в порядке следования в машинном коде, а в порядке готовности к выполнению

— основано на алгоритме Томасуло и его развитии

- переименование регистров (англ. register renaming)

— способ реализации параллелизма на уровне команд

- переименование регистров основано на таком же подходе

— между архитектурными регистрами (x86: eax, ebx, ...) и физическими нет строгого соответствия

◦ при этом физических регистров существенно больше

— при исполнении на CPU архитектурному регистру назначается не постоянная физическая копи (!)

Версия 1	Версия 2
r1 = m[1024]	r1 = m[1024]
r1 += 2	r1 += 2
m[1032] = r1	m[1032] = r1
r1 = m[2048]	r2 = m[2048]
r1 += 4	r2 += 4
m[2056] = r1	m[2056] = r2

Технология склейки микроопераций (Micro-fusion)

Объединение в одну микрооперацию несколько зависимых микроопераций при этом результирующая микрооперация должна порождать один результат при внеочередном исполнении нет смысла разделять зависимые операции, т.к. они не могут выполняться параллельно, а всегда будут выполняться последовательно

Наличие нескольких зависимых микроопераций приводит к тому, что станции резервирования и ячейки ROB используются впустую

Объединенная микрооперация будет занимать одну станцию резервирования и одну ячейку ROB

Для поддержки технологии необходимо, чтобы станция резервирования могла хранить дескрипторы для трех и более операндов

После стадии декодирования должно находиться устройство, которое выполняет склейку

Ошибки в микрооперациях

Intel Pentium (1994 г.)

-вещественный сопроцессор

-ошибка в таблицах поиска (пример):

4195835 * 3145727 / 3145727 = 4195579

Skylake / Kaby Lake :

"Under complex micro-architectural conditions, short loops of less than 64 instructions that use AH, BH, CH or DH registers as well as their corresponding wider register (e.g. RAX, EAX or AX for AH) may cause unpredictable system behavior. This can only happen when both logical processors on the same physical processor are active.“ "Intel Skylake / Kaby Lake processors: broken hyper-threading“ (сообщество Debian)

25. Спекуляция. Спекулятивный суперскалярный процессор. Основные этапы исполнения инструкций

Сокращения:

- ФУ - функциональное устройство
- СР - станция резервирования
- БУ - буфер упорядочивания
- ССП - спекулятивный суперскалярный процессор
- **что-то там** - к вопросу вроде как не относится, но было в ответах прошлых лет

Спекуляция - выполнение некоторой операции, необходимость выполнения которой носит вероятностный характер.

Суперскалярный процессор способен заканчивать или отставлять 2 и более команды за такт

ССП выполняет все инструкции вне порядка (out of order) и спекулирует над инструкциями условного перехода и инструкциями загрузки/сохранения

ССП способен выполнять инструкции, которые следуют за невыполненной инструкцией условного перехода, основываясь на данных предсказателя ветвления, и в случае неверного предсказания выполняется переход.

Этапы исполнения инструкции ССП:

1. Планирование инструкции:

- Планирование инструкций осуществляется с вершины очереди планирования.
- Для выполнения планирования должны быть доступны:
 - станция резервирования на требуемом ФУ
 - ячейка в БУ

Если хотя бы один из ресурсов не доступен, то инструкция ожидает в очереди.

2. Выборка инструкции

3. Ожидание готовности операндов:

- Проверяется существование операндов в регистровом файле. Если они существуют, то записываются в СР.
- Если операндов нет в регистровом файле, то проверяется их существование в БУ. Если операнды есть в БУ, то они записываются в СР.
- Если и в БУ операнды не найдены, то в СР записывается номер ячейки БУ, в которой содержится инструкция для вычисления этого операнда(-ов).
- В файл переименования для результирующего регистра инструкции помещается номер ячейки БУ, которая ее содержит.

4. Исполнение инструкции:

- a. Инструкции запускаются на исполнении по мере готовности их операндов и доступности ФУ.
- b. По общей шине передается результат инструкции и номер ячейки БУ, которая ее содержит.

5. Запись результата:

- a. Запись результата производится одновременно во все ожидающие СР и БУ.
- b. Инструкция загрузки выполняется в две стадии:
 - i. Вычисление адреса.
 - ii. Чтение из памяти. Запускается если нет конфликтов с предыдущими инструкциями загрузки и сохранения.
 - iii. Инструкция сохранения на этапе исполнения вычисляет адрес операнда и ожидает готовности результата, запись в память произойдет на стадии завершения.

6. Завершение (commit):

- a. Инструкция после вычисления результата ожидает в БУ своего завершения.
- b. Завершаются инструкции, которые находятся на вершине БУ. БУ поддерживает поведение очереди FIFO.
- c. На этапе завершения:
 - Для арифметических инструкций и инструкций загрузки проверяется наличия исключений и если исключений нет, то результат записывается регистровый файл.
 - Для инструкции сохранения проверяется наличия исключений и если исключений нет, то результат записывается в память.
 - Для инструкции условного перехода проверяется правильно она была предсказана или нет, если да, то инструкция удаляется из БУ, если нет, происходит сброс процессора: очищаются все стадии, на планирование отправляется первая инструкция из правильной ветви.

Изменения в аппаратной схеме:

- 1. Добавлен буфер упорядочивания (БУ) (record buffer).
 - a. Используется для:
 - i. Хранения результатов работы инструкций между стадией записи результата и стадией завершения.
 - ii. Поддержание программного порядка инструкций на стадии завершения.
 - b. В состав БУ входят:
 - i. Выполняемая инструкция. Это может быть арифметическая операция, загрузка, сохранение, условный переход и другие.
 - ii. Адрес результата.
 - iii. Поля для временного хранения результата выполнения инструкции.
- 2. Удален буфер сохранения.

Ограничения спекуляции:

1. Если инструкция в спекулятивном режиме вызвало кэш промах стоит его обрабатывать или нет?
Обрабатываются обычно только самые легкие события, такие как промах в кэш первого уровня, остальные события откладываются до выхода инструкции из спекулятивного режима.
2. Ошибочная спекуляция приводит к выполнению бессмысленной работы и трате энергии.
Количество инструкций условного перехода над которыми производится спекуляция может быть ограничена.

Спекуляция по управлению:

- **Спекуляция по управлению** - возможность корректно исполнять инструкции в ветви, для которой еще не вычислено условие.
- Количество инструкций условного перехода над которыми производится спекуляция может быть ограничена.
- Поддержка спекуляции по управлению включена в состав архитектуры EPC.

Спекуляция по данным:

- Спекулятивный суперскалярный процессор может маскировать большую латентность инструкций загрузки за счет внеочередного исполнения других инструкций.
- EPC процессор обрабатывает последовательности независимых инструкций в программном порядке и поэтому не может пропустить зависимые инструкции.

Чтобы скрыть латентность инструкций загрузки он должен выполнять их как можно раньше. Для этого компилятор должен перенести инструкцию загрузки как можно выше по коду, относительно инструкции которая от нее зависит.

26. Алгоритм Томасуло. Планирование инструкций

Томасуло - алгоритм динамического планирования инструкций, который позволяет исполнять инструкции в порядке отличном от программного. При одновременном исполнении двух и более инструкций позволяет разрешить RAW (read after write), и устранить WAR (write after read) и WAW (write after write) конфликты

- Достоинства:
 - повышение пропускной способности
 - уменьшение времени простоя процессора
- Недостатки:
 - большие аппаратные затраты на реализацию дополнительных устройств

(синим текстом - для лучшего понимания с, писать необязательно)

Состав процессора:

- очередь планирования (Instruction queue)[ОП]
- регистровый файл (FP Registers) [РФ]
- станции резервирования (Reserve Station) [СР]
- простое вещественное устройство (FP adder)
- сложное вещественное устройство (FP Multiplier)
- общая шина данных (Common Data Bus)[ОШД]
- устройство вычисления адреса (Address Unit)
- буфера загрузки (Load buffer)
- буфера сохранения (Store buffer)
- устройство работы с памятью (Memory Unit)

Планирование инструкций:

1 Выборка с вершины ОП

- выборка происходит по 1 инструкции за такт
- выборка осуществляется в программном порядке, так как очередь FIFO

2 Декодирование

- назначение на исполнительное устройство
- если все СР устройства заняты, то инструкция возвращается в ОП и ожидает освобождения RS

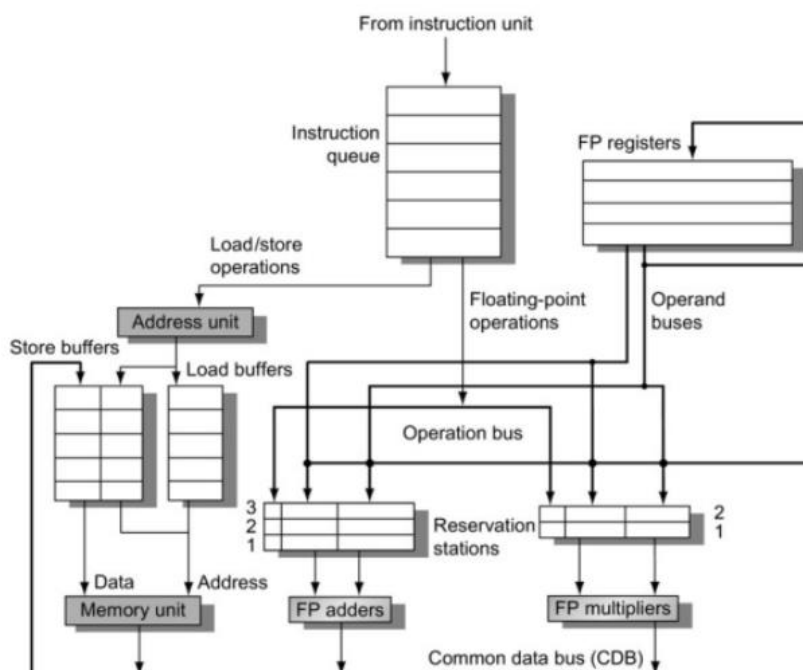
3 Выборка операндов

- если операнды вычислены, то они выбираются из РФ, если нет то в дескрипторах СР устанавливается ссылки на другие СР

27. Алгоритм Томасуло. Состав процессора. Этапы исполнения инструкций

Состав процессора:

- очередь планирования (Instruction queue)[ОП]
- регистровый файл (FP Registers) [РФ]
- станции резервирования (Reserve Station) [СР]
- простое вещественное устройство (FP adder)
- сложное вещественное устройство (FP Multiplier)
- общая шина данных (Common Data Bus)[ОШД]
- устройство вычисления адреса (Address Unit)
- буфера загрузки (Load buffer)
- буфера сохранения (Store buffer)
- устройство работы с памятью (Memory Unit)



Этапы исполнения:

Классический конвейер:

- выборка инструкции
- выборка операндов
- исполнение
- сохранение результата

Томасуло:

- выборка инструкции
- планирование инструкции
- ожидание готовности операндов
- исполнение
- сохранение результата

Обработка инструкций загрузки и сохранения

- инструкция загрузки:
 - вычисление адреса
 - выполнение загрузки по адресу
- инструкция сохранения:
 - вычисление адреса
 - ожидания готовности операнда
 - выполнение сохранения по адресу
- буфер загрузки: поле адреса
- буфер сохранения: поле адреса, СР для операнда

Ожидание готовности операндов, исполнение, сохранение результатов

- инструкция ожидает в СР до тех пор, пока не будут вычислены все ее операнды и записаны в соответствующие дескрипторы
- передача вычисленного операнда происходит по ОШД вместе с номером СР, который его содержал
- каждая СР слушает ОШД и сравнивает значение номер передаваемого по ней СР с ожидаемым. Если номера совпадают то она забирает значение операнда с ОШД
- передаваемые по ОШД данные сохраняются в регистровый файл
- если все операнды находятся СР, то инструкция отправляется на исполнение

Порядок исполнения инструкций загрузки и сохранения

- определяется наличием зависимостей между инструкциями по ячейкам памяти
- адреса ячеек вычисляются на первом этапе
- инструкция загрузки ожидает завершения всех предшествующих инструкций сохранения по данному адресу
- инструкция сохранения ожидает завершения всех предшествующих инструкций загрузки и сохранения по данному адресу

28. Спекулятивный суперскалярный процессор

Спекуляция – выполнение некоторой операции, необходимость выполнения которой носит вероятностный характер.

ССП выполняет все инструкции вне порядка (out of order) и спекулирует над инструкциями условного перехода и инструкциями загрузки/сохранения

ССП - способен выполнять инструкции, которые следуют за невыполненной инструкцией условного перехода, основываясь на данных предсказателя ветвления, и в случае неверного предсказания выполнять откат.

Этапы исполнения инструкции:

1. Выборка инструкции
2. Планирование инструкции
3. Ожидание готовности операндов
4. Исполнение
5. Запись результата
6. Завершения (commit)

Изменения в аппаратной схеме:

- Добавлен буфер упорядочивания (для хранения результатов инструкций между стадией записи результата и стадией завершения) (содержит выполняемую инструкцию и адрес результата инструкции).
- Поля для временного хранения результата инструкции
- Убран буфер сохранения

Этап планирования:

- Инструкции планируются сверху очереди.
- Инструкция выполняется, если есть свободный ресурс (СР) и ячейка в блоке управления (БУ). Если ресурсов нет – инструкция ждёт.
- Операнды выбираются в три этапа:
 - Проверка регистров – если данные есть, они копируются в СР.
 - Если в регистрах нет – проверка БУ. Данные копируются, если найдены.
 - Если нигде нет – записывается номер ячейки БУ, которая вычислит операнд

Этап исполнения и записи результата:

- Инструкции выполняются, когда готовы операнды и есть свободный ФУ.
- Результат и номер ячейки БУ передаются по общей шине.

- Запись идёт сразу во все ожидающие СР и БУ.
- Загрузка выполняется в два этапа:
 - Адрес вычисляется.
 - Данные читаются из памяти (если нет конфликтов с другими загрузками/сохранениями).

Этап завершения:

- Инструкция ждёт завершения в БУ после вычисления результата.
- Завершаются инструкции сверху очереди БУ (FIFO).
- Если не спекуляция, то проверка на исключения. Если их нет — запись в регистры.
- Если условный переход, то проверка предказания. Если верно — удаление из БУ, если нет — сброс процессора и запуск с правильной ветви

Ограничения спекуляции:

- Ошибочная спекуляция тратит ресурсы и энергию, НО кол-во инструкций с условным переходом может быть ограничено.
- Инструкция в спекулятивном режиме может вызвать кэш-промах, НО обрабатываются только самые легкие события (например, промах L1), а остальные откладываются до выхода из спекулятивного режима.

29. EPIC. Механизмы поддержки спекуляции

EPIC (англ. *explicitly parallel instruction computing* — «вычисление с явным параллелизмом машинных команд»)

- Призвана сохранить достоинства VLIW архитектуры и устранить ее недостатки.
- Достоинства VLIW и EPIC: возможность параллельного исполнения инструкций задается на уровне архитектуры, а не определяется динамически в процессе исполнения.
- Недостаток VLIW/ преимущества EPIC: решена проблема бинарной совместимости между различными процессорами.

(синим текстом - для лучшего понимания, писать необязательно)

• *VLIW:*

- параллельное выполнение инструкции задается в рамках одного пакета.
- количество инструкций в одном пакете определяется количеством исполняемых устройств в процессоре.

• *EPIC:*

- параллельно выполняемые инструкции задаются специальными шаблонами в пакете.
- количество параллельно исполняемых инструкций может быть больше количества инструкций в пакете.
- количество инструкций в пакете определяется архитектурой и не зависит от аппаратной реализации.

Проблема бинарной совместимости:

- Программы, скомпилированные для одного процессора, могут не работать оптимально или вовсе не работать на другом процессоре с другим количеством функциональных устройств (ФУ).
- В архитектуре EPIC (включая Itanium) задача назначения инструкций на ФУ перекладывается на процессор, а не на компилятор.
- Процессор сам определяет, какие инструкции запускать на каком устройстве, учитывая своё текущее количество ФУ и их доступность.
- Это означает, что программа не "зависит" от конфигурации конкретного процессора, что обеспечивает бинарную совместимость.

Примеры (??)

- разработки архитектуры следующего за VLIW поколения начались в компании HP в конце 80-ых годов.
- в 1993 году разработана первая EPIC архитектура PlayDoh
- в 1997 году анонсирована архитектура Intel IA-64 (Intel и HP)
- в 2001 выпущен первый процессор Intel Itanium
- в 2002 выпущен процессор Intel Itanium 2

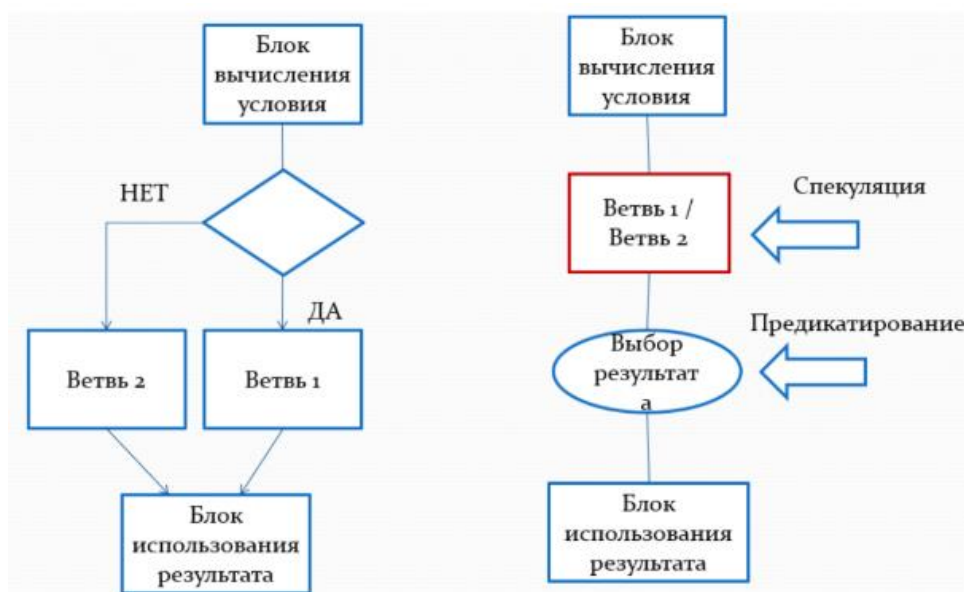
Спекулятивное выполнение — это метод оптимизации, при котором компьютерная система выполняет некоторую задачу, которая может оказаться ненужной. Работа выполняется до того, как станет известно, действительно ли она необходима, чтобы

предотвратить задержку, которая может возникнуть при выполнении работы после того, как станет известно, что она необходима. Если выясняется, что работа в конце концов не нужна, большинство изменений, внесённых в работу, отменяется, а результаты игнорируются.

Механизмы поддержки спекуляции:

• **Спекуляция по управлению**

- это возможность корректно исполнять инструкции в ветви, для которой еще не вычислено условие.
 - процессоры с архитектурой RISC и CISC выполняют спекуляцию на аппаратном уровне. Первоначально такой возможности в эти архитектуры не закладывалось.
 - поддержка спекуляция по управлению включена в состав архитектуры EPIC
- спекуляция по управлению в EPIC основано на:
- наличие двух типов инструкций: спекулятивных и не спекулятивных;
 - параллельном исполнении спекулятивных инструкций из двух ветвей;
 - использование предикатирования для выбора результат корректной ветви;
 - использование третьего состояния регистров для сохранения поведения исключений во время спекуляции;
 - все исключения генерируемые спекулятивными инструкциями являются отложенными;
 - спекулятивная инструкция, сгенерировавшая отложенное исключение, помещает в свой регистр результата значение NaT (Not a Thing) (реализуется добавлением дополнительного бита к каждому регистру процессора; спекулятивными являются только те инструкции, которые сохраняют данные в регистры с поддержкой состояния NaT; не спекулятивные инструкции сохраняют результаты в память или обычные регистры)



Программная спекуляция по управлению в EPIC:

- достоинства:
 - маленькие аппаратные затраты на реализацию
 - нет затрат на неверное предсказание

- недостатки:

- требует исполнения большего количества инструкций
- проигрывает по скорости аппаратной спекуляции в случае низкой вероятности неверного предсказания перехода

- **Спекуляция по данным**

Это выполнение инструкции загрузки данных (чтения из памяти) раньше времени, даже если процессор ещё не знает, понадобится ли результат этой загрузки или нет.

Процессор не может просто "перепрыгнуть" через инструкции, которые зависят друг от друга.

Чтобы ускорить выполнение, процессор или компилятор должны попытаться загрузить данные раньше, но при этом нельзя нарушить порядок выполнения: например, нельзя прочитать данные, которые ещё не были записаны.

"Компилятор переносит загрузку вверх"

Компилятор старается вставить инструкцию загрузки (чтения) как можно раньше в коде, до того, как данные понадобятся. Это помогает "скрыть" время ожидания загрузки данных.

Проблема совпадения адресов

Если в коде сначала идёт запись (st) по адресу, а потом чтение (ld) с того же адреса, их нельзя менять местами. Компилятор должен проверить, совпадают ли адреса записи и чтения. Но эта задача слишком сложная для компилятора (он не всегда может заранее узнать адреса), поэтому её решают на уровне процессора.

Решение на стороне процессора

- Компилятор всё равно переносит загрузку вверх, как будто конфликта адресов нет.

- Перед использованием загруженных данных процессор вставляет специальную инструкцию проверки.

- Если произошёл конфликт (данные изменились после загрузки), процессор может перезапустить загрузку данных или сгенерировать исключение для исправления ситуации.

30. EPIC. Пакет инструкций – способ явного задания параллелизма уровня команд

Параллелизм уровня команд (Instruction Level Parallelism, ILP) – это потенциальное перекрытие нескольких инструкций во времени. Благодаря ILP такие инструкции могут выполняться параллельно, повышая производительность.

Базовый блок – это последовательность инструкций, имеющая одну точку входа и одну точку выхода. Параллелизм присутствует среди инструкций базового блока.

Для получения существенного прироста производительности необходимо использовать параллелизм не только внутри, но и между базовыми блоками.

Простейший способ увеличения ILP – использование параллелизма итераций цикла. Разворачивание циклов (Loop Unrolling) – увеличение числа инструкций базового блока (копирование тела цикла несколько раз).

Определение степени зависимости между инструкциями – критический фактор для определения уровня параллелизма в программе. Независимые инструкции могут выполняться одновременно. Зависимые должны выполняться по порядку, однако они могут частично перекрываться во времени.

Зависимость – свойство программы.

Конфликт, возникший (или не возникший) из зависимости – свойство организации архитектуры.

Существует 3 типа зависимостей:

- Зависимости по данным.
- Зависимости по именам.
- Зависимости по управлению.

31. Технология OpenMP

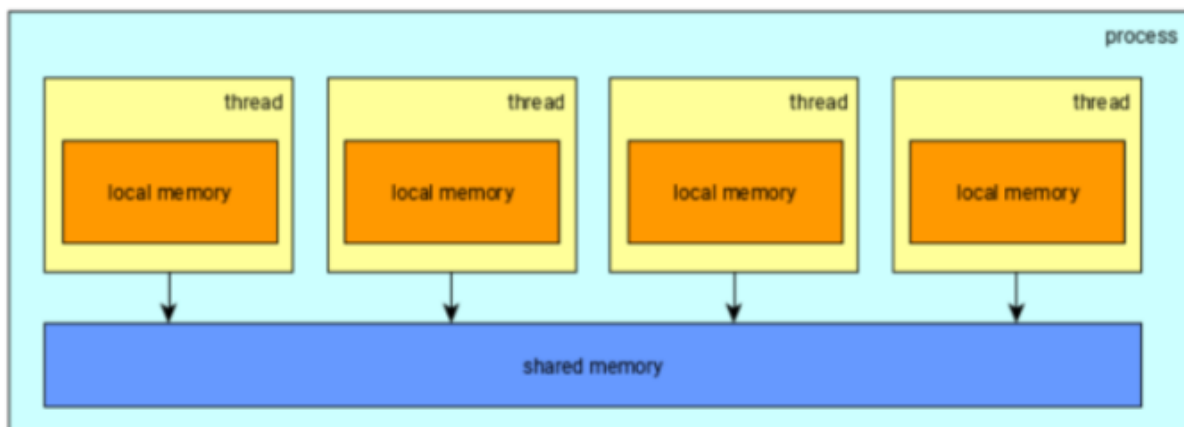
OpenMP – библиотека для параллельного программирования вычислительных систем с общей памятью.

Поддерживаемый языки программирования:

C, C++, Fortran

Поддержка версий:

- актуальная версия – OpenMP 5.2
- MS Visual Studio – OpenMP 2.0 (!!!) планов по актуализации нет
- GCC 6 – базовая поддержка OpenMP 4.5
- GCC 11 – ожидается полная поддержка OpenMP 4.5 и частичная поддержка OpenMP 5.0



Формат директив:

```
#pragma omp <директива> [раздел [ [,] раздел]...]
```

директивы:

- parallel
- for
- parallel for
- section или sections
- single
- master
- critical
- flush
- ordered
- atomic

parallel - распараллеливание потока в отмеченной директивой области видимости.

```
// A - single thread
#pragma omp parallel
{
    // B - many threads
}
```

parallel for - директива распараллеливания цикла

```
#pragma omp parallel for
for (i=0; i < 100; i++)
{
    temp = array[i];
    array[i] = do_something(temp);
}
```

*опция `private` позволяет сделать внешнюю общую переменную приватной, т.е. значения переменной между потоками никак не связаны.

reduction

Формат директивы: `reduction(оператор: список)`

Возможные операторы — "+", "*", "-", "&", "|", "^", "&&", "||".

Список — перечисляет имена общих переменных. У переменных должен быть скалярный тип (например, `float`, `int` или `long`, но не `std::vector`, `int []` и т. д.).

Принцип работы:

Для каждой переменной создаются локальные копии в каждом потоке.

Локальные копии инициализируются соответственно типу оператора. Для аддитивных операций — 0 или его аналоги, для мультипликативных операций — 1 или ее аналоги.

Над локальными копиями переменных после выполнения всех операторов параллельной области выполняется заданный оператор. Порядок выполнения операторов не определен.

планирование циклов

```
#pragma omp parallel for schedule(kind [, chunk size])
```

два основных режима планирования:

статический

- `schedule(static)`
- `schedule(static, 10)`

динамический

- `schedule(dynamic)`
- `schedule(dynamic, 10)`

1. Тип static:

Если размер не указан, все множество итераций цикла делится на примерно равные части (по числу входящих в них итераций) в соответствии с числом потоков и каждому потоку назначается своя часть. Минимальные затраты на планирование, но возможна несбалансированность нагрузки по потокам.

Размер указан. В этом случае все множество итераций цикла делится на части указанного размера (по числу входящих в них итераций) и полученные части назначаются потокам циклически.

2. Тип dynamic. Части множества итераций (количество итераций в каждой части равно указанному размеру, а если он не указан, то 1 по умолчанию) назначаются каждому потоку по запросу. Т.е., что после начального распределения частей очередная часть достается тому, кто раньше закончил свою работу. Хорошая балансировка нагрузки по потокам, но большие временные затраты на планирование (поскольку операция запроса потока на очередную порцию итераций цикла и его удовлетворения требует относительно больших временных затрат).

Директива sections

Директива `omp sections` распределяет работу между потоками, привязанными к определенной параллельной области.

```
#pragma omp parallel
{
    #pragma omp for
    for (i=0; i<x; i++)
        Fn1();
    #pragma omp sections
    {
        #pragma omp section
        {
            TaskA();
        }
        #pragma omp section
        {
            TaskB();
        }
        #pragma omp section
        {
            TaskC();
        }
    }
}
```

Также существуют директивы `master` для указания области исполнения главным потоком без определения критерия главного потока вручную и `nowait`, не производящий синхронизацию потоков в конце блока.

Примеры функций omp:

`int omp_get_num_threads(void);` позволяет получить количество потоков, задействованных для выполнения данной задачи. (возвращает 1 вне многопоточной секции)

`int omp_set_num_threads(int NumThreads);` устанавливает число потоков, которое будет создано при входе в параллельную секцию.

`int omp_get_num_procs(void);` возвращает число доступных физических потоков (учитывая многопоточность ядра процессора).